

SECTION A: QUESTIONS

◆ *Understanding Algorithms & Running Time*

1. A search engine must return results instantly from millions of entries. Which algorithm type should be preferred?
 2. A sorting system processes small datasets frequently. Should it use Merge Sort or Insertion Sort?
 3. A system processes data that doubles every hour. Why is $O(n^2)$ unsuitable?
-

◆ *Rate of Growth & Big-O*

4. An algorithm takes $2n + 5$ steps. What is its complexity?
 5. A nested loop runs $n \times n$ times. What is complexity?
 6. An algorithm halves input each step. Identify complexity.
 7. Compare $O(n \log n)$ vs $O(n^2)$ for large datasets.
-

◆ *Recursion*

8. A factorial calculation is implemented recursively. What is its time complexity?
 9. A recursive Fibonacci solution recalculates values repeatedly. What issue arises?
 10. Why does recursion require a base case?
 11. A function calls itself twice per step. Predict complexity.
-

◆ *Backtracking*

12. A Sudoku solver tries all possibilities but abandons invalid ones early. Which technique is used?
 13. In N-Queens problem, why prune branches early?
 14. A password generator tests all combinations with constraints. Which approach is used?
 15. Why is backtracking slower than greedy?
-

◆ *Dynamic Programming*

[Type here]

16. Fibonacci is optimized using memoization. What technique is used?
 17. A system stores results of subproblems to avoid recomputation. What concept is this?
 18. Knapsack problem requires optimal selection under constraints. Which approach is suitable?
 19. What are the two key properties of dynamic programming?
 20. Why is dynamic programming faster than recursion?
-

◆ ***Greedy Algorithms***

21. A coin change system selects the largest denomination first. Which strategy is used?
 22. In activity selection, tasks with earliest finish time are chosen. Why?
 23. Why doesn't greedy always work for the knapsack problem?
 24. A network uses minimum spanning tree for cost reduction. Which algorithms are used?
-

◆ ***Higher-Order / Case-Based***

25. A routing system must choose the fastest path dynamically. DP or Greedy?
26. A scheduling system must consider all possibilities to find the optimal solution. Which method is used?
27. Design an algorithm to minimize cost with overlapping subproblems. Which paradigm is used?
28. A brute-force solution is too slow. How would you optimize it?
29. Compare recursion vs dynamic programming in terms of performance.
30. You must design an algorithm for real-time decisions with minimal computation. Which approach is best?